

Pruebas de software

¿Qué es una prueba de software?

Una prueba de software es la ejecución controlada de un programa con el objetivo de detectar defectos: comprobar si cumple los requisitos especificados y si se comporta como se espera.

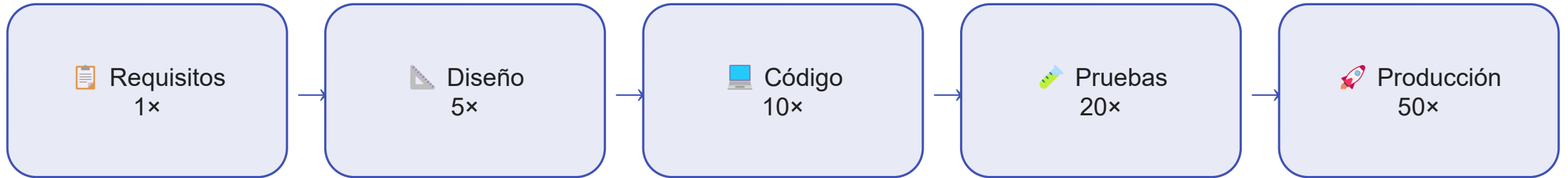
"La prueba de programas puede utilizarse para mostrar la presencia de errores, pero nunca para demostrar su ausencia."
— Edgar Dijkstra

Pasar las pruebas no significa que el programa esté libre de fallos, solo que esas pruebas concretas no han encontrado ninguno. El objetivo real es reducir al mínimo los fallos que llegan al usuario.

⚠ Las pruebas no son solo una comprobación al final de la implementación. Deben hacerse en distintas etapas y no se limitan a la funcionalidad: también evalúan rendimiento, seguridad, usabilidad y compatibilidad.

¿Por qué cuesta tanto detectar un error tarde?

El coste de arreglar un fallo depende de cuándo se descubre. Cuanto más tarde aparece, más trabajo hay que deshacer y rehacer:



Un error en producción puede costar hasta 50 veces más de solucionar que el mismo error detectado en la fase de requisitos.

Por eso las pruebas no son un trámite al final del proyecto: representan entre el 30 % y el 50 % del coste total. Detectar un error de diseño a tiempo —como olvidar un campo clave en la base de datos— puede ahorrarte rehacer trabajo que ya se daba por terminado.

Recomendaciones de G.J. Myers (1 / 2)

G.J. Myers recogió hábitos que marcan la diferencia entre probar bien y probar para cumplir el expediente:

Recomendación	En qué consiste
Definir resultados esperados	Cada caso debe indicar de antemano qué resultado se espera. La discrepancia entre lo esperado y lo obtenido señala el defecto.
Separación de roles	El programador no debería probar su propio código: otra persona detecta errores más fácilmente porque no comparte los mismos puntos ciegos.
Inspección minuciosa	Hay que revisar con atención los resultados. Ignorar un síntoma raro "porque seguro que no es nada" deja pasar defectos reales.
Amplitud en los datos	Probar con datos válidos y con datos inválidos, para cubrir tanto el caso esperado como el inesperado.
Doble enfoque	Comprobar que el software hace lo que debería, pero también que no hace lo que no debería (efectos secundarios no deseados).

Recomendaciones de G.J. Myers (2 / 2)

Recomendación	En qué consiste
Documentar los casos	Guardar los casos de prueba para poder repetirlos más adelante, en vez de improvisarlos y desecharlos.
Asumir que hay defectos	Partir de la base de que siempre existen errores y dedicar recursos suficientes a buscarlos.
Concentración de defectos	Los errores tienden a agruparse: si encuentras uno en una zona del código, es probable que haya más cerca.
Creatividad	Probar bien es una tarea creativa, no mecánica. Requiere ingenio para encontrar el mayor número de defectos con los recursos disponibles.

💡 Probar el propio código es difícil porque quien lo escribió tiende a usarlo de la forma "correcta". Alguien externo lo usa de formas que el autor no anticipó, y ahí es donde aparecen los errores.

Verificación y validación

Verificación

¿Lo estamos construyendo bien?

El software cumple las especificaciones técnicas tal como fueron definidas.

Validación

¿Estamos construyendo lo correcto?

El software cumple las necesidades reales del usuario, aunque eso no estuviera en la especificación.

La metáfora de la casa

Verificación

Revisar durante la construcción que los planos se siguen: ¿los muros tienen la altura especificada?, ¿la instalación eléctrica cumple la normativa?

Validación

Preguntarle al propietario, una vez construida, si la casa cumple sus expectativas: ¿las habitaciones tienen el tamaño que necesitaba?

⚠ Un software puede estar perfectamente verificado y aun así no validar, si la especificación no recogía lo que el cliente realmente quería.

Niveles de prueba

Las pruebas se organizan en niveles según qué parte del sistema se comprueba. La pirámide indica cuántas pruebas hay de cada tipo: muchas en la base (rápidas y baratas), pocas en la cima (lentas y caras).



Nivel	Qué comprueba
Unidad	Que un método o clase funciona de forma aislada
Integración	Que los componentes combinados respetan el diseño
Validación	Que el sistema completo cumple los requisitos definidos
Sistema	El funcionamiento global, incluyendo red, otros sistemas, etc.

Tipos de pruebas: caja negra y caja blanca

La clasificación más fundamental distingue si el probador tiene acceso o no al código interno del programa:

■ Caja blanca (estructural)

El probador tiene acceso al código fuente.

Evalúa la estructura interna: rutas de ejecución, condiciones, bucles.

Técnicas: camino básico, cobertura de sentencias y de rutas.

→ Ver: Técnicas de diseño de casos de prueba

■ Caja negra (funcional)

El probador solo ve entradas y salidas.

Evalúa el comportamiento según la especificación, sin mirar el código.

Técnicas: partición equivalente, análisis de valores límite, aleatorias.

→ Ver: Técnicas de diseño de casos de prueba

Caja blanca y caja negra son enfoques complementarios, no excluyentes. Se usan juntos para cubrir distintos tipos de errores.

Tipos de pruebas: rendimiento y regresión

Pruebas de rendimiento

Evalúan cómo se comporta el sistema cuando la carga aumenta, no si hace lo correcto:

Tipo	Qué evalúa
De carga	Comportamiento con el volumen esperado: usuarios concurrentes, transacciones por segundo.
De estrés	Resistencia en condiciones extremas, aumentando la carga hasta que el sistema falla.
De estabilidad	Si la aplicación soporta carga continuada sin fugas de memoria ni pérdida de rendimiento.
De picos	Cómo reacciona ante cambios bruscos en el número de usuarios.

Otras

Tipo	Qué evalúa
Aleatorias	Genera casos a partir de modelos estadísticos para descubrir fallos en escenarios imprevisibles.
De regresión	Que los cambios recientes no han roto funcionalidades que antes funcionaban correctamente.

Documentación de incidencias

Encontrar un defecto no es el final del proceso. Hay que documentarlo para que alguien pueda reproducirlo, entenderlo y solucionarlo sin investigarlo desde cero. A esto se le llama incidencia (bug report o issue).

Un informe de incidencia útil incluye, como mínimo:

Campo	Qué debe recoger
Descripción	Qué ocurre, en una frase clara.
Pasos para reproducir	La secuencia exacta de acciones que provocan el fallo.
Resultado esperado vs. obtenido	Qué debería pasar y qué pasa realmente.
Entorno	Sistema operativo, versión del programa, datos de entrada relevantes.
Severidad / prioridad	No todos los errores son igual de urgentes.

En la práctica esto se registra en herramientas como Jira o GitHub Issues, donde cada incidencia queda asignada con su estado visible para todo el equipo.

Incidencias: informe vago vs. informe útil

La diferencia entre un informe que alguien puede usar y uno que obliga a investigar desde cero:

✗ Informe vago — no sirve

Descripción: El login a veces no funciona.

Pasos: Entrar y logarse.

Resultado: No entra.

¿Qué falla aquí? "A veces" no es reproducible. "No entra" no dice si hay mensaje de error, si redirige o si no ocurre nada. Quien recibe esto tiene que empezar a investigar desde cero.

✓ Informe útil — reproducible

Descripción: El login acepta contraseña incorrecta con espacio al final.

Pasos: 1. Ir a `http://localhost:8080/login`
2. Usuario: admin
3. Contraseña: "1234 " (espacio al final)
4. Pulsar Entrar

Esperado: mensaje de error "Credenciales incorrectas".

Obtenido: acceso concedido al panel de admin.

Entorno: Windows 11, Chrome 124, v2.3.1.

Severidad: Alta — permite acceso no autorizado.